

Producto y arquitectura: app de gastos con tagueo automatico

Documento de Producto y Arquitectura Técnica

App de control de gastos con tagueo automático (individual y hogar)

Preparado como Product Manager y Full Stack Developer para el proyecto de Franco.

1. Resumen ejecutivo

La idea es una aplicación de registro de gastos donde el usuario escribe (o dicta) el gasto en lenguaje natural —por ejemplo “30000 pesos hamburguesa en Guido”— y un motor interno lo categoriza automáticamente. A diferencia de las apps que ya existen en el mercado, esta propuesta agrega una capa que hoy nadie resuelve bien: **múltiples miembros de un mismo hogar cargando gastos desde su propio dispositivo, con una vista agregada del gasto total de la casa**, sin que el foco sea dividir deudas entre personas (eso ya lo hacen Splitwise, Tricount, Splid).

El hueco de mercado es real: las apps de tagueo inteligente (MonAi, Kuantto) son de un solo usuario. Las apps multiusuario (Splitwise, Tricount) no tienen tagueo inteligente y su objetivo es “quién le debe a quién”, no “cuánto gastó la familia este mes”. Nadie combina ambas cosas con foco en visibilidad familiar en vez de liquidación de deudas.

2. Investigación de mercado

2.1 MonAi

Qué es: app de finanzas personales, solo iOS, con entrada por texto o voz y categorización automática por IA. Guarda los datos en iCloud del usuario (no hay backend propio ni cuenta).

Ventajas: - Entrada por voz, muy rápida de usar en el momento del gasto. - Reconoce fecha, descripción, monto y categoría en un solo texto. - Interfaz minimalista, cuidada, bien valorada por los usuarios. - No pide credenciales ni datos personales; todo queda en el dispositivo del usuario vía iCloud, lo cual es un buen argumento de privacidad. - Tiene una función de “listas compartidas” con otra persona (una aproximación parcial a lo multiusuario, aunque no está pensada como panorama de hogar). - Precio de suscripción bajo comparado con otras apps del rubro.

Desventajas: - Solo iOS. Si en tu casa hay un Android, ya queda afuera. - Depende de iCloud: no hay un backend centralizado, lo que complica justamente el caso de “varias personas viendo lo mismo” salvo con la función limitada de listas compartidas. - Reseñas de usuarios reportan que la IA falla con transferencias entre cuentas propias y necesita ajuste manual constante. - Diseño “minimalista a propósito”: varios usuarios piden más reportes mensuales, comparativas y recomendaciones, y el equipo reconoce que hoy no las tiene. - No está pensada para agregación por hogar; la unidad de análisis es siempre la persona.

2.2 Kunto

Qué es: fintech chilena (lanzada en 2025) que categoriza gastos automáticamente procesando los correos que envían los bancos (comprobantes, resúmenes de cuenta), en vez de depender de que el usuario tipee cada gasto.

Ventajas: - Automatización real: no hace falta cargar nada a mano si el gasto pasa por una cuenta bancaria conectada. - Aprende de las correcciones del usuario con el tiempo. - Fuerte mensaje de privacidad: aseguran que nunca venden los datos. - Cierre de mes preciso porque procesa el estado de cuenta completo, no solo notificaciones sueltas. - Ya están desarrollando una función de “finanzas en pareja”, lo que confirma que el mercado está pidiendo justamente lo multiusuario.

Desventajas: - Depende de la integración con bancos específicos (arrancó con Banco de Chile, Santander y Falabella); fuera de esos bancos y de Chile, hoy no sirve. Para Argentina, en el estado actual, no sería utilizable. - Suscripción paga desde el vamos (no hay plan gratuito permanente). - Al depender de parsing de correos bancarios, la app no sirve bien para gastos en efectivo, que en Argentina siguen siendo una parte grande del gasto diario (el propio caso que planteaste, la hamburguesa, podría ser efectivo). - La función de “finanzas en pareja” está en desarrollo, no lanzada: confirma la oportunidad pero también significa que todavía no hay un competidor consolidado ahí.

2.3 Cómo funciona el tagueo automático (estado del arte)

Investigando cómo resuelven esto las apps existentes y qué técnicas se usan en la industria, hay tres enfoques, de más simple a más sofisticado, y normalmente se combinan:

1. **Reglas por palabra clave:** un diccionario de palabras asociadas a categorías (“nafta” → transporte, “netflix” → suscripciones). Es la base de casi cualquier categorizador; barato de implementar, pero no generaliza a palabras que no están en el diccionario.
2. **Coincidencia difusa con historial propio (fuzzy matching):** cada vez que el usuario corrige o confirma una categoría, esa combinación texto-categoría se guarda. Ante un gasto nuevo, se busca el texto más parecido en el historial (con librerías como rapidfuzz o thefuzz) y se sugiere esa categoría. Esto es lo que le da a la app la sensación de “aprender” de cada usuario, que es justamente lo que Kunto destaca como diferencial y lo que los usuarios de MonAi piden que mejore.

3. **Embeddings semánticos:** el texto se convierte en un vector numérico que representa su significado (con un modelo liviano, no hace falta un modelo de lenguaje grande), y se compara por similitud contra categorías ya etiquetadas. La ventaja frente al enfoque anterior es que generaliza: “medialunas” y “panadería” caen en la misma categoría aunque no compartan palabras, cosa que el fuzzy matching puro no logra.

La arquitectura más razonable para este proyecto es una combinación: reglas para los casos obvios y de arranque en frío, más aprendizaje incremental por historial (fuzzy matching) como corazón del sistema, con posibilidad de sumar embeddings más adelante si se necesita mejor generalización.

2.4 Conclusión de la investigación (gap de mercado)

	Tageo automático por texto libre	Multiusuario / hogar	Enfoque en agregación (no en deudas)	Disponible en Argentina hoy
MonAi	Sí	Parcial (listas compartidas)	Sí	Sí (solo iOS)
Kuanto	Sí (vía banco)	En desarrollo	Sí	No
Splitwise / Tricount / Splid	No	Sí	No (foco en deudas)	Sí
Producto propuesto	Sí	Sí, nativo	Sí	Sí, diseñado para pesos argentinos y efectivo

Ninguna solución actual cubre las cuatro columnas a la vez. Ese es el espacio disponible.

3. Visión del producto

Propuesta de valor: una app donde cargar un gasto toma cinco segundos escribiendo una frase natural, y donde cada persona de la casa ve automáticamente cuánto gastó la familia entera, sin tener que reconciliar Excel ni preguntarse quién le debe a quién.

Diferenciales: - Tageo automático por texto libre, funciona con o sin conexión bancaria (clave para efectivo, muy relevante en Argentina). - Modo individual y modo hogar en la misma cuenta, sin fricción para pasar de uno a otro. - El hogar es una vista agregada de gasto, no un sistema de deudas: se responde “¿cuánto gastamos este mes?”, no “¿quién le debe a quién?”. - Categorías y aprendizaje del motor de tageo compartidos dentro del hogar: si uno enseña que “medialunas” es “comida”, los demás miembros se benefician de esa corrección.

4. Usuarios y personas

Persona 1 — El organizador del hogar (ej. la mamá o el papá)

Paga los gastos grandes y recurrentes (préstamo del auto, expensas, servicios). Quiere ver el panorama completo de la casa por mes y por año, y detectar si algún gasto se está yendo de rango. No tiene tiempo ni ganas de aprender una herramienta compleja.

Persona 2 — El estudiante/joven independiente (perfil de Franco)

Gasta en el día a día (comida, salidas, transporte) y quiere cargar rápido, sin fricción, mientras está en la calle. Le importa que la categorización sea automática porque no quiere perder tiempo eligiendo categorías a mano. Le interesa ver su propio gasto individual además del gasto de la casa.

Persona 3 — El usuario ocasional/escéptico

Ya probó otras apps de finanzas y las abandonó porque le generaban trabajo extra o ansiedad (este patrón aparece explícitamente en las reseñas de Kuantos y MonAi). Necesita que la app funcione bien desde el primer uso, sin curva de aprendizaje.

5. Funcionalidades

5.1 MVP (versión mínima viable)

- Registro de gasto por texto libre (monto + descripción en una sola frase).
- Motor de tagueo automático por reglas + aprendizaje por historial (fuzzy matching).
- Corrección manual de categoría, que retroalimenta al motor.
- Modo individual: dashboard de gasto propio por semana / mes.
- Modo hogar: crear o unirse a un hogar (código de invitación), y ver el gasto total de todos los miembros combinado, con desglose por persona y por categoría.
- Objetivos de gasto (por categoría o total) con aviso al superarlos.
- Historial de gastos con búsqueda y filtro por persona, categoría y período.

5.2 Roadmap (post-MVP)

- Entrada por voz (dictado), tal como hace MonAi.
 - Notificaciones/recordatorios de carga diaria.
 - Gastos recurrentes automáticos (alquiler, suscripciones) que no haga falta tipear cada mes.
 - Reportes comparativos mes a mes y tendencias (justo lo que los usuarios de MonAi piden y hoy no tienen).
 - Multi-moneda, útil si alguien del hogar cobra en dólares.
 - Conexión opcional con resúmenes bancarios (estilo Kuantos) como método adicional de carga, no como único método.
 - Exportar a CSV/Excel.
-

6. Flujos clave (historias de usuario)

1. Como usuario, quiero escribir “30000 pesos hamburguesa en Guido” y que la app cargue el gasto con categoría “Comida” sin que yo tenga que elegirla.
2. Como usuario, quiero corregir una categoría mal asignada una sola vez y que la app no vuelva a equivocarse con descripciones parecidas.
3. Como miembro de un hogar, quiero ver cuánto gastamos entre

- todos este mes, con el detalle de cuánto puso cada uno.
4. Como usuario, quiero fijar un objetivo mensual (propio o del hogar) y recibir un aviso si lo supero.
 5. Como usuario nuevo, quiero crear un hogar o unirme al que ya creó un familiar con un código simple, sin fricción.
-

7. Modelo de datos conceptual

- **usuarios:** id, nombre, email, hash de contraseña, fecha de alta.
 - **hogares:** id, nombre, fecha de creación, código de invitación.
 - **usuario_hogar:** relación muchos a muchos entre usuarios y hogares (un usuario puede pertenecer a más de un hogar, o a ninguno).
 - **gastos:** id, usuario_id, hogar_id (nulo si es gasto puramente individual sin hogar), monto, texto_original, categoria_id, fecha, medio_de_pago (opcional).
 - **categorias:** id, nombre, hogar_id (nulo si es una categoría global del sistema, o asociada a un hogar si es personalizada).
 - **reglas_tageo:** id, hogar_id o usuario_id, texto_o_patron, categoria_id (esta tabla es la que alimenta el fuzzy matching: cada corrección del usuario queda acá).
 - **objetivos:** id, usuario_id o hogar_id, categoria_id (nulo si es objetivo total), monto_limite, periodo (semanal/mensual).
-

8. Stack tecnológico propuesto

Esta sección elige, entre las opciones disponibles hoy, las que dan mejor rendimiento, mejor experiencia de usuario final y mayor velocidad de desarrollo — sin condicionarlo a herramientas ya conocidas por el equipo.

8.1 Mobile (la app real, para vos, tu papá y tu mamá)

- **React Native con Expo, usando la New Architecture (Fabric + JSI).** Es el estándar de la industria en 2026 para llegar a iOS y Android con un solo código base, con una brecha de rendimiento frente a nativo que hoy es prácticamente imperceptible para una app de este tipo. Esto además resuelve de raíz la principal debilidad de MonAi (ser solo iOS).
- **TypeScript** en todo el proyecto, no JavaScript plano: atrapa errores antes de que lleguen a producción y permite compartir tipos entre el backend y el frontend.
- **Expo Router** para la navegación, **Reanimated 3** para animaciones fluidas a 60fps, **NativeWind** (Tailwind adaptado a React Native) para estilos consistentes y rápidos de escribir.
- **TanStack Query (React Query)** para el manejo de datos remotos: cachea, revalida y sincroniza el estado de los gastos entre pantallas sin lógica manual.
- **Expo Application Services (EAS)** para compilar y publicar en las dos tiendas sin necesitar una Mac propia, y para actualizaciones over-the-air (subir cambios sin pasar por la revisión de Apple/Google cada vez).
- **Notificaciones push** con Expo Push (que internamente usa FCM y APNs) para los recordatorios y avisos de objetivos superados.

8.2 Web (dashboard complementario, opcional pero recomendable)

- **Next.js** (App Router), que hoy es el framework de referencia para aplicaciones web rápidas con React. Sirve para un dashboard más cómodo de ver en una pantalla grande, sin ser la app principal.
- Comparte componentes y lógica con la app mobile al vivir ambos en el mismo monorepo (ver 8.5).

8.3 Backend y datos

- **Base de datos: PostgreSQL**, gestionada por un proveedor como Supabase o Neon. PostgreSQL es el estándar para datos relacionales como los de esta app (gastos, categorías, hogares) y ambos proveedores ofrecen capa gratuita generosa para empezar.
- **Capa de API: tRPC** en vez de una API REST tradicional. Al estar todo en TypeScript de punta a punta, tRPC comparte los tipos entre backend y frontend automáticamente: si cambiás un campo en el servidor, el error aparece al instante en la app, sin documentación de API que se desactualice. Esto acelera mucho el desarrollo cuando el equipo es chico (o una sola persona).
- **ORM: Drizzle**, más liviano y con mejor rendimiento que alternativas como Prisma, con control fino sobre las consultas SQL cuando hace falta.
- **Autenticación: Supabase Auth o Clerk**. Resuelven de fábrica login, sesiones multi-dispositivo, recuperación de contraseña y (si más adelante lo querés) login social, sin tener que programar y mantener esa parte a mano.
- **Sincronización en tiempo real**: Supabase ofrece actualizaciones en vivo sobre PostgreSQL, útil para que si tu mamá carga un gasto, tu dashboard del hogar se actualice al instante sin recargar.

8.4 Motor de tagueo automático

- **Fase 1 (arranque)**: reglas por palabra clave más búsqueda por similitud de texto (librerías tipo fuse.js en el mismo backend TypeScript), alimentada por las correcciones que cada usuario va haciendo.
- **Fase 2 (mejor generalización)**: incorporar una función de embeddings semánticos para comparar el significado de las descripciones y no solo el texto literal, corriendo como una función serverless separada del resto del backend para no afectar el rendimiento de la API principal.
- El aprendizaje sigue siendo compartido dentro del hogar: una corrección de categoría de cualquier miembro mejora el tagueo para todos.

8.5 Estructura del proyecto

- **Monorepo con Turborepo**, con la app mobile (Expo), la web (Next.js) y el backend (tRPC) conviviendo en un solo repositorio, compartiendo tipos, validaciones y lógica de negocio sin duplicar código.

8.6 Calidad, monitoreo e infraestructura

- **Sentry** para monitoreo de errores en producción, **PostHog** para analítica de producto (qué pantallas usa más cada persona, dónde se traba el flujo de carga de un gasto).
- **Jest** más **Testing Library** para pruebas automatizadas del motor de tagueo, que es la pieza más sensible a errores silenciosos.
- **Hosting**: Vercel para el backend/web (despliegue instantáneo, capa gratuita amplia) y EAS para la distribución mobile.

8.7 Seguridad y privacidad (punto crítico en una app financiera)

- Autenticación delegada a un proveedor especializado (Supabase Auth/Clerk) en vez de manejar contraseñas a mano, reduciendo el riesgo de errores de seguridad caseros.
 - Comunicación siempre por HTTPS/TLS.
 - Reglas de acceso a nivel de fila en PostgreSQL (row-level security) para que cada usuario solo pueda ver el detalle de gasto que el hogar decida permitir, con el agregado siempre visible para todos los miembros.
-

9. Plan de trabajo sugerido (fases de desarrollo)

1. **Semana 1:** monorepo con Turborepo, esquema de base de datos en PostgreSQL con Drizzle, autenticación con Supabase Auth, primeros endpoints tRPC de alta y consulta de gastos.
 2. **Semana 2:** motor de tagueo por reglas y similitud de texto, con endpoint de corrección de categoría que alimente el aprendizaje compartido del hogar.
 3. **Semana 3:** lógica de hogares (crear, unirse, agregación en tiempo real) y pantallas de dashboard (semana/mes/trimestre/año) en la app Expo.
 4. **Semana 4:** objetivos de gasto, notificaciones push, pulido de UI con NativeWind/Reanimated, y build de prueba con EAS para instalar la app real en el teléfono de tu papá y tu mamá.
-

10. Riesgos a tener en cuenta

- **Adopción familiar:** la app solo funciona si tu papá y tu mamá realmente la usan a diario. Vale la pena que el flujo de carga sea más simple que escribir un WhatsApp.
- **Privacidad dentro del hogar:** decidir de entrada si todos ven el detalle de todos o solo el agregado, para evitar fricciones familiares no técnicas.
- **Arranque en frío del motor de tagueo:** las primeras cargas de cada hogar van a tener más correcciones manuales hasta que el sistema aprenda; conviene comunicarlo así desde el principio para no generar la misma frustración que reportan los usuarios de MonAi.